

263 Project

Hannah Hamling

2026-04-30

Read in data/variable selection

```
library(readxl)
library(dplyr)
fetal.dat <- read_excel("../Data/raw.fetal.data.xlsx")
fetal.dat <- fetal.dat[-1,] #empty first row
#each row is its own id
#variables we'll actually use (including NSP)
#removing date, id, filename, FEB, b, e, AC, DR, SUSP, class, and other classification variables not in
fetal <- fetal.dat |>
  dplyr::select(-c("FileName","Date","SegFile","b", "e", "LBE", "AC", "DR", "A",
                  "B", "C", "D", "E", "AD","DE","LD","FS","SUSP","CLASS"))
#make NSP a factor (only categorical variable in our selected features)
fetal$NSP <- factor(fetal$NSP)

dim(fetal) #2129 rows (people), 20 features, 1 outcome variable (NSP)
```

```
## [1] 2129  21
```

```
#2 rows missing data on every variable: can remove
#last row missing some variables, doesn't have a classification label
fetal <- fetal[-c(2127,2128),]
dim(fetal) #2127 people, 20 features, 1 outcome variable
```

```
## [1] 2127  21
```

```
fetal[!complete.cases(fetal), ] #only this one row missing data
```

```
## # A tibble: 1 x 21
##      LB      FM      UC  ASTV  MSTV  ALTV  MLTV    DL    DS    DP Width  Min  Max
##    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1    NA   564    23    87     7    91  50.7   16     1     4    NA   NA   NA
## # i 8 more variables: Nmax <dbl>, Nzeros <dbl>, Mode <dbl>, Mean <dbl>,
## #   Median <dbl>, Variance <dbl>, Tendency <dbl>, NSP <fct>
```

Look at class frequencies (for later class minority adjustment)

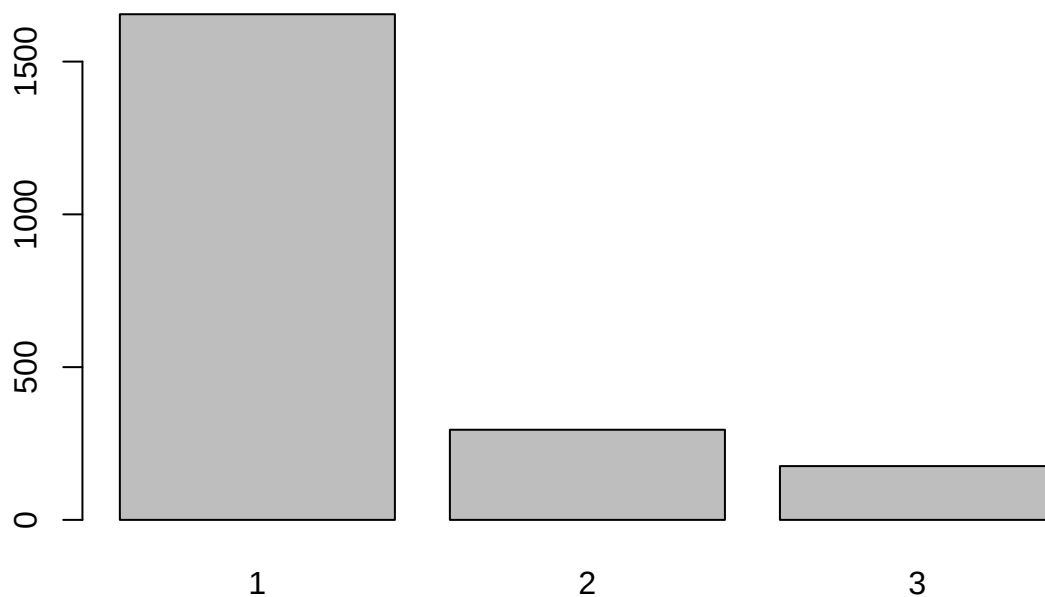
```
summary(fetal$NSP)
```

```
##      1      2      3 NA's  
## 1655  295  176      1
```

```
#1655 are normal, 295 are suspect, 176 are pathologic, 1 missing  
#plot(fetal$NSP)
```

```
# drop the 1 observation with NA for NSP -> 2126 observations  
fetal <- fetal |> drop_na(NSP)
```

```
#1655 are normal, 295 are suspect, 176 are pathologic  
plot(fetal$NSP)
```



```
library(caret)
```

```
## Loading required package: lattice
```

```
##
```

```
## Attaching package: 'caret'
```

```
## The following object is masked from 'package:purrr':
```

```
##
```

```
## lift
```

```
set.seed(263)

train_indices <- createDataPartition(fetal$NSP, p = 0.8, list = FALSE)

fetal_train <- fetal[train_indices, ]
fetal_test  <- fetal[-train_indices, ]

# make sure there are some of each NSP in both train and test data
table(fetal_train$NSP)
```

```
##
##      1      2      3
## 1324   236   141
```

```
table(fetal_test$NSP)
```

```
##
##      1      2      3
## 331    59    35
```

Custom Asymmetric Loss Analysis

We have a classification problem with 3 classes of fetal state (1 = Normal, 2 = Suspect, 3 = Pathological). Classes 2 and 3 are underrepresented compared to class 1. Clinically, certain misclassifications are more harmful than others.

- Mistaking 3 as 1: Worst possible mistake
- Mistaking 2 as 1: Also a pretty bad mistake
- Mistaking 3 as 2: Bad, but not as bad as the previous two cases
- Mistaking 2 as 3: not that bad
- Mistaking 1 as 2: not that bad
- Mistaking 1 as 3: not that bad

Thus, we can define a custom loss that punishes certain mixups accordingly.

```
# row is true class, column is predicted class

# all misclassifications are punished equally
loss_default <- matrix(c(0, 1, 1,
                        1, 0, 1,
                        1, 1, 0), nrow = 3, byrow = TRUE)

# we can change these values, like idk is a false negative 5x as bad as a false positive
loss_mild <- matrix(c(0, 1, 1,
                     4, 0, 1,
                     5, 2, 0), nrow = 3, byrow = TRUE)

loss_aggressive <- matrix(c(0, 1, 1,
                           7, 0, 1,
```

```

      10, 2, 0), nrow = 3, byrow = TRUE)

calculate_loss <- function(true_labels, predicted_labels, loss_matrix) {
  true_labels <- as.numeric(true_labels)
  predicted_labels <- as.numeric(predicted_labels)
  mean(mapply(FUN = function(t, p) loss_matrix[t, p], true_labels, predicted_labels))
}

```

Default Single Tree Using All Variables

Let's see how a single basic tree with all variables and default arguments does with our custom loss. We see that 2 gets misclassified as 1 a lot (bad mistake), which increases the loss.

```

library(rpart)
library(rpart.plot)

```

```
## Warning: package 'rpart.plot' was built under R version 4.5.2
```

```

set.seed(263)

basic_tree <- rpart(NSP ~ . , data = fetal_train, method = "class")

#rpart.plot(basic_tree)

# training confusion matrix
basic_tree_train_prediction <- predict(basic_tree, fetal_train, type = "class")
confusionMatrix(basic_tree_train_prediction, fetal_train$NSP) # 93.36 train acc

```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction    1    2    3
##           1 1304   78    8
##           2   11  153    2
##           3    9    5  131
##
## Overall Statistics
##
##               Accuracy : 0.9336
##               95% CI : (0.9207, 0.9449)
##       No Information Rate : 0.7784
##       P-Value [Acc > NIR] : < 2.2e-16
##
##               Kappa : 0.8065
##
## Mcnemar's Test P-Value : 3.332e-11
##
## Statistics by Class:
##
##               Class: 1 Class: 2 Class: 3
## Sensitivity      0.9849  0.64831  0.92908

```

```
## Specificity          0.7719  0.99113  0.99103
## Pos Pred Value      0.9381  0.92169  0.90345
## Neg Pred Value      0.9357  0.94593  0.99357
## Prevalence          0.7784  0.13874  0.08289
## Detection Rate      0.7666  0.08995  0.07701
## Detection Prevalence 0.8172  0.09759  0.08524
## Balanced Accuracy    0.8784  0.81972  0.96005
```

```
# test confusion matrix
```

```
basic_tree_test_prediction <- predict(basic_tree, fetal_test, type = "class")
confusionMatrix(basic_tree_test_prediction, fetal_test$NSP) # 92.47 test acc
```

```
## Confusion Matrix and Statistics
```

```
##
##           Reference
## Prediction  1    2    3
##           1 326  24    2
##           2   5  35    1
##           3   0   0   32
```

```
##
## Overall Statistics
```

```
##
##           Accuracy : 0.9247
##           95% CI : (0.8954, 0.9479)
##           No Information Rate : 0.7788
##           P-Value [Acc > NIR] : 4.783e-16
##
##           Kappa : 0.7755
```

```
##
## Mcnemar's Test P-Value : 0.001471
```

```
##
## Statistics by Class:
```

```
##
##           Class: 1 Class: 2 Class: 3
## Sensitivity      0.9849  0.59322  0.91429
## Specificity      0.7234  0.98361  1.00000
## Pos Pred Value   0.9261  0.85366  1.00000
## Neg Pred Value   0.9315  0.93750  0.99237
## Prevalence       0.7788  0.13882  0.08235
## Detection Rate   0.7671  0.08235  0.07529
## Detection Prevalence 0.8282  0.09647  0.07529
## Balanced Accuracy 0.8541  0.78841  0.95714
```

```
# model predicts a lot of 2's as 1's, so we should punish this more
```

```
# training losses
```

```
calculate_loss(basic_tree_train_prediction, fetal_train$NSP, loss_default) # default, 0.0664
```

```
## [1] 0.06643151
```

```
calculate_loss(basic_tree_train_prediction, fetal_train$NSP, loss_mild) # mild, 0.11
```

```
## [1] 0.1099353
```

```
calculate_loss(basic_tree_train_prediction, fetal_train$NSP, loss_aggressive) # aggressive, 0.1558
```

```
## [1] 0.1557907
```

```
# test losses
```

```
calculate_loss(basic_tree_test_prediction, fetal_test$NSP, loss_default) # default, 0.075
```

```
## [1] 0.07529412
```

```
calculate_loss(basic_tree_test_prediction, fetal_test$NSP, loss_mild) # mild, 0.111
```

```
## [1] 0.1105882
```

```
calculate_loss(basic_tree_test_prediction, fetal_test$NSP, loss_aggressive) # aggressive, 0.146
```

```
## [1] 0.1458824
```

Default RF

The default RF model shows a large class error rate for class 2 (and 3 somewhat), so the loss increases when we use our custom loss.

```
library(randomForest)
```

```
## randomForest 4.7-1.2
```

```
## Type rfNews() to see new features/changes/bug fixes.
```

```
##
```

```
## Attaching package: 'randomForest'
```

```
## The following object is masked from 'package:dplyr':
```

```
##
```

```
##      combine
```

```
## The following object is masked from 'package:ggplot2':
```

```
##
```

```
##      margin
```

```
set.seed(263)
```

```
rf_default <- randomForest(NSP ~ ., data = fetal_train)
```

```
rf_default # once again, 2 is being misclassified as 1, OOB error = 5.7%
```

```
##
```

```
## Call:
```

```
## randomForest(formula = NSP ~ ., data = fetal_train)
```

```
##           Type of random forest: classification
```

```
##               Number of trees: 500
## No. of variables tried at each split: 4
##
##           OOB estimate of  error rate: 5.7%
## Confusion matrix:
##      1   2   3 class.error
## 1 1310  11   3  0.01057402
## 2   60 172   4  0.27118644
## 3    8  11 122  0.13475177
```

```
# train losses
calculate_loss(predict(rf_default, fetal_train), fetal_train$NSP, loss_default) # default, 0.00118
```

```
## [1] 0.001175779
```

```
calculate_loss(predict(rf_default, fetal_train), fetal_train$NSP, loss_mild) # mild, 0.00294
```

```
## [1] 0.002939447
```

```
calculate_loss(predict(rf_default, fetal_train), fetal_train$NSP, loss_aggressive) # aggressive, 0.0047
```

```
## [1] 0.004703116
```

```
# test losses
calculate_loss(predict(rf_default, fetal_test), fetal_test$NSP, loss_default) # default, 0.0588
```

```
## [1] 0.05882353
```

```
calculate_loss(predict(rf_default, fetal_test), fetal_test$NSP, loss_mild) # mild, 0.0964
```

```
## [1] 0.09647059
```

```
calculate_loss(predict(rf_default, fetal_test), fetal_test$NSP, loss_aggressive) # aggressive, 0.132
```

```
## [1] 0.1317647
```

Default Boosting

Boosting model also misclassifies 2 as 1 a lot. 3 as 1 too sometimes.

```
library(gbm)
```

```
## Warning: package 'gbm' was built under R version 4.5.2
```

```
## Loaded gbm 2.2.3
```

```
## This version of gbm is no longer under development. Consider transitioning to gbm3, https://github.com
```

```

set.seed(263)
gbm_default <- gbm(NSP ~ ., data = fetal_train)

## Distribution not specified, assuming multinomial ...

## Warning: Setting `distribution = "multinomial"` is ill-advised as it is
## currently broken. It exists only for backwards compatibility. Use at your own
## risk.

# training confusion matrix
gbm_default_train_prediction <- as.factor(max.col((predict.gbm(gbm_default, fetal_train, type = "response"))))

## Using 100 trees...

confusionMatrix(gbm_default_train_prediction, fetal_train$NSP) # 94.42 train acc

## Confusion Matrix and Statistics
##
##              Reference
## Prediction    1     2     3
##           1 1299    53    12
##           2   16   182     4
##           3    9    1   125
##
## Overall Statistics
##
##              Accuracy : 0.9442
##              95% CI : (0.9322, 0.9546)
##      No Information Rate : 0.7784
##      P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.8417
##
##  McNemar's Test P-Value : 6.311e-05
##
## Statistics by Class:
##
##              Class: 1 Class: 2 Class: 3
## Sensitivity          0.9811   0.7712   0.88652
## Specificity          0.8276   0.9863   0.99359
## Pos Pred Value       0.9523   0.9010   0.92593
## Neg Pred Value       0.9258   0.9640   0.98978
## Prevalence           0.7784   0.1387   0.08289
## Detection Rate       0.7637   0.1070   0.07349
## Detection Prevalence 0.8019   0.1188   0.07937
## Balanced Accuracy    0.9044   0.8788   0.94006

# test confusion matrix
gbm_default_test_prediction <- as.factor(max.col((predict.gbm(gbm_default, fetal_test, type = "response"))))

## Using 100 trees...

```



```
confusionMatrix(gbm_default_test_prediction, fetal_test$NSP) # 92.47 test acc
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   1    2    3
##           1 322  18    2
##           2   9  41    3
##           3   0   0   30
##
## Overall Statistics
##
##           Accuracy : 0.9247
##           95% CI : (0.8954, 0.9479)
##           No Information Rate : 0.7788
##           P-Value [Acc > NIR] : 4.783e-16
##
##           Kappa : 0.785
##
## Mcnemar's Test P-Value : 0.04601
##
## Statistics by Class:
##
##           Class: 1 Class: 2 Class: 3
## Sensitivity      0.9728  0.69492  0.85714
## Specificity      0.7872  0.96721  1.00000
## Pos Pred Value   0.9415  0.77358  1.00000
## Neg Pred Value   0.8916  0.95161  0.98734
## Prevalence       0.7788  0.13882  0.08235
## Detection Rate   0.7576  0.09647  0.07059
## Detection Prevalence 0.8047  0.12471  0.07059
## Balanced Accuracy 0.8800  0.83106  0.92857
```

```
# train losses
```

```
calculate_loss(gbm_default_train_prediction, fetal_train$NSP, loss_default) # default, 0.0558
```

```
## [1] 0.0558495
```

```
calculate_loss(gbm_default_train_prediction, fetal_train$NSP, loss_mild) # mild, 0.1058
```

```
## [1] 0.1058201
```

```
calculate_loss(gbm_default_train_prediction, fetal_train$NSP, loss_aggressive) # aggressive, 0.1605
```

```
## [1] 0.1604938
```

```
# test losses
```

```
calculate_loss(gbm_default_test_prediction, fetal_test$NSP, loss_default) # default, 0.0752
```

```
## [1] 0.07529412
```

```
calculate_loss(gbm_default_test_prediction, fetal_test$NSP, loss_mild) # mild, 0.1388
```

```
## [1] 0.1388235
```

```
calculate_loss(gbm_default_test_prediction, fetal_test$NSP, loss_aggressive) # aggressive, 0.202
```

```
## [1] 0.2023529
```

RF CV

now doing cv to select the best RF hyperparameters but according to our custom loss (hopefully helps to fix the bad sorts of mixups)

Hyperparameters include: mtry and ntree

```
# warning this chunk takes TIME
set.seed(263)
# 5 folds
K <- 5
fold <- sample(1:K, nrow(fetal_train), replace = TRUE)

mtry_values <- c(3, 5, 7, 9) # we have 20 variables, the default is p/3
ntree_values <- c(250, 500, 750, 1000, 1500) # default is 500

rf_cv_results <- data.frame(mtry = c(), ntree = c(), loss_default = c(), loss_mild = c(), loss_aggressive = c())

for(m in mtry_values) {
  for(n in ntree_values) {
    fold_loss_default <- c()
    fold_loss_mild <- c()
    fold_loss_aggressive <- c()

    for(k in 1:K) {
      train <- fetal_train[fold != k, ]
      val <- fetal_train[fold == k, ]

      rf_model <- randomForest(NSP ~ .,
                              data = train,
                              mtry = m,
                              ntree = n)

      fold_loss_default[k] <- calculate_loss(val$NSP, predict(rf_model, val), loss_default)
      fold_loss_mild[k] <- calculate_loss(val$NSP, predict(rf_model, val), loss_mild)
      fold_loss_aggressive[k] <- calculate_loss(val$NSP, predict(rf_model, val), loss_aggressive)
    }

    rf_cv_results <- rf_cv_results |> rbind(data.frame(
      mtry = m,
      ntree = n,
      loss_default = mean(fold_loss_default),
      loss_mild = mean(fold_loss_mild),
      loss_aggressive = mean(fold_loss_aggressive)))
  }
}
```

```
)
}
}
```

```
rf_cv_results
```

```
##      mtry ntree loss_default loss_mild loss_aggressive
## 1      3   250   0.06471110 0.2033978      0.3404209
## 2      3   500   0.06953737 0.2152796      0.3593001
## 3      3   750   0.06944408 0.2184347      0.3662417
## 4      3  1000   0.06892663 0.2158124      0.3609621
## 5      3  1500   0.06828350 0.2151149      0.3602102
## 6      5   250   0.06137966 0.1884803      0.3174143
## 7      5   500   0.06243638 0.1952216      0.3252268
## 8      5   750   0.06127334 0.1929911      0.3254686
## 9      5  1000   0.06134463 0.1937743      0.3281063
## 10     5  1500   0.06073776 0.1878915      0.3157845
## 11     7   250   0.06137947 0.1914493      0.3228314
## 12     7   500   0.06082086 0.1874633      0.3154181
## 13     7   750   0.06077279 0.1873028      0.3151451
## 14     7  1000   0.06134199 0.1912993      0.3225690
## 15     7  1500   0.06192950 0.1884595      0.3163018
## 16     9   250   0.06253638 0.1924937      0.3237634
## 17     9   500   0.06196699 0.1925143      0.3249639
## 18     9   750   0.06141450 0.1897144      0.3193266
## 19     9  1000   0.06255696 0.1932167      0.3257788
## 20     9  1500   0.06141450 0.1897144      0.3193266
```

```
# this is the best set of hyperparameters for the mild loss
# we see a lower class error in classes 2 and 3 compared to the default og RF model
# still kinda bad for class 2 tho
randomForest(NSP ~., data = fetal_train, ntree = 750, mtry = 7)
```

```
##
## Call:
## randomForest(formula = NSP ~ ., data = fetal_train, ntree = 750,      mtry = 7)
##              Type of random forest: classification
##              Number of trees: 750
## No. of variables tried at each split: 7
##
##              OOB estimate of  error rate: 5.41%
## Confusion matrix:
##           1    2    3 class.error
## 1 1304   15    5 0.01510574
## 2   57  175    4 0.25847458
## 3    4    7  130 0.07801418
```

GBM CV

now doing cv to select the best GBM hyperparameters but according to our custom loss (hopefully helps to fix the bad sorts of mixups)

Hyperparameters include: n.trees, interaction.depth, shrinkage

```
gbm_cv_results <- data.frame(n.trees = c(), interaction.depth = c(), shrinkage = c(), loss_default = c(),
                              loss_mild = c(), loss_aggressive = c())

n.trees_values <- c(50, 100, 250, 500, 1000) # default is 100
interaction.depth_values <- c(1, 2, 3) # default is a depth of 1
shrinkage_values <- c(0.001, 0.01, 0.1, 0.2) # default is 0.1

for(n in n.trees_values) {
  for(d in interaction.depth_values) {
    for(s in shrinkage_values) {
      fold_loss_default <- c()
      fold_loss_mild <- c()
      fold_loss_aggressive <- c()

      for(k in 1:K) {
        train <- fetal_train[fold != k, ]
        val <- fetal_train[fold == k, ]

        gbm_model <- gbm(NSP ~ ., data = train, distribution = "multinomial",
                          n.trees = n, interaction.depth = d, shrinkage = s)

        val_probs <- predict(gbm_model, val, n.trees = n, type = "response")[,1]
        colnames(val_probs) <- c("1", "2", "3")
        val_predicted_label <- colnames(val_probs)[max.col(val_probs)]

        fold_loss_default[k] <- calculate_loss(val$NSP, val_predicted_label, loss_default)
        fold_loss_mild[k] <- calculate_loss(val$NSP, val_predicted_label, loss_mild)
        fold_loss_aggressive[k] <- calculate_loss(val$NSP, val_predicted_label, loss_aggressive)
      }
      gbm_cv_results <- gbm_cv_results |> rbind(data.frame(
        n.trees = n,
        interaction.depth = d,
        shrinkage = s,
        loss_default = mean(fold_loss_default),
        loss_mild = mean(fold_loss_mild),
        loss_aggressive = mean(fold_loss_aggressive))
      )
    }
  }
}

gbm_cv_results
```

##	n.trees	interaction.depth	shrinkage	loss_default	loss_mild	loss_aggressive
## 1	50	1	0.001	0.13685292	0.4180816	0.6856236
## 2	50	1	0.010	0.13395138	0.4415169	0.7451350
## 3	50	1	0.100	0.07422822	0.2469617	0.4279639
## 4	50	1	0.200	0.07458045	0.2205293	0.3724025
## 5	50	2	0.001	0.10729015	0.3225960	0.5364286
## 6	50	2	0.010	0.09459656	0.3210277	0.5523732
## 7	50	2	0.100	0.06591304	0.2077379	0.3549345
## 8	50	2	0.200	0.06678229	0.1936025	0.3246484
## 9	50	3	0.001	0.09220552	0.3029478	0.5156623

## 10	50	3	0.010	0.08402950	0.2827246	0.4862122
## 11	50	3	0.100	0.06640036	0.1992748	0.3374630
## 12	50	3	0.200	0.06191544	0.1788810	0.2994681
## 13	100	1	0.001	0.14031189	0.4260633	0.6992743
## 14	100	1	0.010	0.12419196	0.4358119	0.7473901
## 15	100	1	0.100	0.06998985	0.2118136	0.3601348
## 16	100	1	0.200	0.06995663	0.1978493	0.3299302
## 17	100	2	0.001	0.10850267	0.3228436	0.5369613
## 18	100	2	0.010	0.09222836	0.3161468	0.5467638
## 19	100	2	0.100	0.06352728	0.1889211	0.3190451
## 20	100	2	0.200	0.06199853	0.1706275	0.2817043
## 21	100	3	0.001	0.09176284	0.2973877	0.5061248
## 22	100	3	0.010	0.07502697	0.2612382	0.4534049
## 23	100	3	0.100	0.06189712	0.1826684	0.3076136
## 24	100	3	0.200	0.06240967	0.1806072	0.3023887
## 25	250	1	0.001	0.13562329	0.4249066	0.7028580
## 26	250	1	0.010	0.09728570	0.3344374	0.5781876
## 27	250	1	0.100	0.06530106	0.1796750	0.2982434
## 28	250	1	0.200	0.06630235	0.1885182	0.3148706
## 29	250	2	0.001	0.10035103	0.3238233	0.5493235
## 30	250	2	0.010	0.07307576	0.2461995	0.4275713
## 31	250	2	0.100	0.06231903	0.1794391	0.2996218
## 32	250	2	0.200	0.06576761	0.1743977	0.2849026
## 33	250	3	0.001	0.08870687	0.2971006	0.5090576
## 34	250	3	0.010	0.06941659	0.2354253	0.4096820
## 35	250	3	0.100	0.06166125	0.1785436	0.2984057
## 36	250	3	0.200	0.05963600	0.1737760	0.2909476
## 37	500	1	0.001	0.13509668	0.4489428	0.7583615
## 38	500	1	0.010	0.07536681	0.2497671	0.4335721
## 39	500	1	0.100	0.06713084	0.1951412	0.3273256
## 40	500	1	0.200	0.06881888	0.1830740	0.3003089
## 41	500	2	0.001	0.09389640	0.3199242	0.5507938
## 42	500	2	0.010	0.06472148	0.2035241	0.3482511
## 43	500	2	0.100	0.05791378	0.1738148	0.2926955
## 44	500	2	0.200	0.06776726	0.1898779	0.3144676
## 45	500	3	0.001	0.08279781	0.2796106	0.4812158
## 46	500	3	0.010	0.06356212	0.1935891	0.3266477
## 47	500	3	0.100	0.06122792	0.1823441	0.3065087
## 48	500	3	0.200	0.06065137	0.1664141	0.2757777
## 49	1000	1	0.001	0.12170622	0.4227362	0.7218144
## 50	1000	1	0.010	0.06825620	0.2123607	0.3635356
## 51	1000	1	0.100	0.07470244	0.1967035	0.3223262
## 52	1000	1	0.200	0.07278816	0.1891942	0.3079926
## 53	1000	2	0.001	0.08924858	0.3029359	0.5209151
## 54	1000	2	0.010	0.06244715	0.1850062	0.3117536
## 55	1000	2	0.100	0.06178364	0.1792498	0.2997268
## 56	1000	2	0.200	0.06311615	0.1790847	0.2980500
## 57	1000	3	0.001	0.07685921	0.2683802	0.4658566
## 58	1000	3	0.010	0.06009218	0.1776828	0.2983050
## 59	1000	3	0.100	0.05836891	0.1676973	0.2806097
## 60	1000	3	0.200	NA	NA	NA

best hyperparameters according to loss mild is 250 trees, depth 3, 0.2 shrinkage
`set.seed(263)`

```
gbm1 <- gbm(NSP ~., data = fetal_train, n.trees = 250, interaction.depth = 3, shrinkage = 0.2)
```

```
## Distribution not specified, assuming multinomial ...
```

```
## Warning: Setting `distribution = "multinomial"` is ill-advised as it is  
## currently broken. It exists only for backwards compatibility. Use at your own  
## risk.
```

```
gbm1_probs_train <- predict(gbm1, fetal_train, n.trees = n, type = "response")[,1]
```

```
## Warning in predict.gbm(gbm1, fetal_train, n.trees = n, type = "response"):  
## Number of trees not specified or exceeded number fit so far. Using 250.
```

```
colnames(gbm1_probs_train) <- c("1", "2", "3")  
gbm1_class_train <- as.factor(colnames(gbm1_probs_train)[max.col(gbm1_probs_train)])  
confusionMatrix(gbm1_class_train, fetal_train$NSP)
```

```
## Confusion Matrix and Statistics
```

```
##
```

```
##           Reference
```

```
## Prediction    1    2    3
```

```
##           1 1322    0    0
```

```
##           2    2  236    0
```

```
##           3    0    0  141
```

```
##
```

```
## Overall Statistics
```

```
##
```

```
##           Accuracy : 0.9988
```

```
##           95% CI : (0.9958, 0.9999)
```

```
## No Information Rate : 0.7784
```

```
## P-Value [Acc > NIR] : < 2.2e-16
```

```
##
```

```
##           Kappa : 0.9968
```

```
##
```

```
## McNemar's Test P-Value : NA
```

```
##
```

```
## Statistics by Class:
```

```
##
```

```
##           Class: 1 Class: 2 Class: 3
```

```
## Sensitivity      0.9985    1.0000    1.00000
```

```
## Specificity      1.0000    0.9986    1.00000
```

```
## Pos Pred Value    1.0000    0.9916    1.00000
```

```
## Neg Pred Value    0.9947    1.0000    1.00000
```

```
## Prevalence        0.7784    0.1387    0.08289
```

```
## Detection Rate    0.7772    0.1387    0.08289
```

```
## Detection Prevalence 0.7772    0.1399    0.08289
```

```
## Balanced Accuracy  0.9992    0.9993    1.00000
```

```
gbm1_probs_test <- predict(gbm1, fetal_test, n.trees = n, type = "response")[,1]
```

```
## Warning in predict.gbm(gbm1, fetal_test, n.trees = n, type = "response"):  
## Number of trees not specified or exceeded number fit so far. Using 250.
```

```
colnames(gbm1_probs_test) <- c("1", "2", "3")
gbm1_class_test <- as.factor(colnames(gbm1_probs_test)[max.col(gbm1_probs_test)])
confusionMatrix(gbm1_class_test, fetal_test$NSP) # OH WOW it really did good on classes 2 and 3 compare
```

```
## Confusion Matrix and Statistics
```

```
##
```

```
##           Reference
```

```
## Prediction    1    2    3
```

```
##           1 323    9    2
```

```
##           2   7   49    2
```

```
##           3    1    1   31
```

```
##
```

```
## Overall Statistics
```

```
##
```

```
##           Accuracy : 0.9482
```

```
##           95% CI : (0.9227, 0.9673)
```

```
## No Information Rate : 0.7788
```

```
## P-Value [Acc > NIR] : <2e-16
```

```
##
```

```
##           Kappa : 0.8572
```

```
##
```

```
## McNemar's Test P-Value : 0.8214
```

```
##
```

```
## Statistics by Class:
```

```
##
```

```
##           Class: 1 Class: 2 Class: 3
```

```
## Sensitivity      0.9758    0.8305    0.88571
```

```
## Specificity      0.8830    0.9754    0.99487
```

```
## Pos Pred Value   0.9671    0.8448    0.93939
```

```
## Neg Pred Value   0.9121    0.9728    0.98980
```

```
## Prevalence       0.7788    0.1388    0.08235
```

```
## Detection Rate   0.7600    0.1153    0.07294
```

```
## Detection Prevalence 0.7859    0.1365    0.07765
```

```
## Balanced Accuracy 0.9294    0.9030    0.94029
```